

Unit 5

Dictionaries



LIST OF CONTENTS

- Operations on Dictionaries
- Dictionary methods
- Using for loop with dictionaries
- Sorting the elements of a dictionary using lambdas
- Converting lists into dictionary
- Converting strings into dictionary
- Passing dictionaries to functions
- Ordered dictionaries

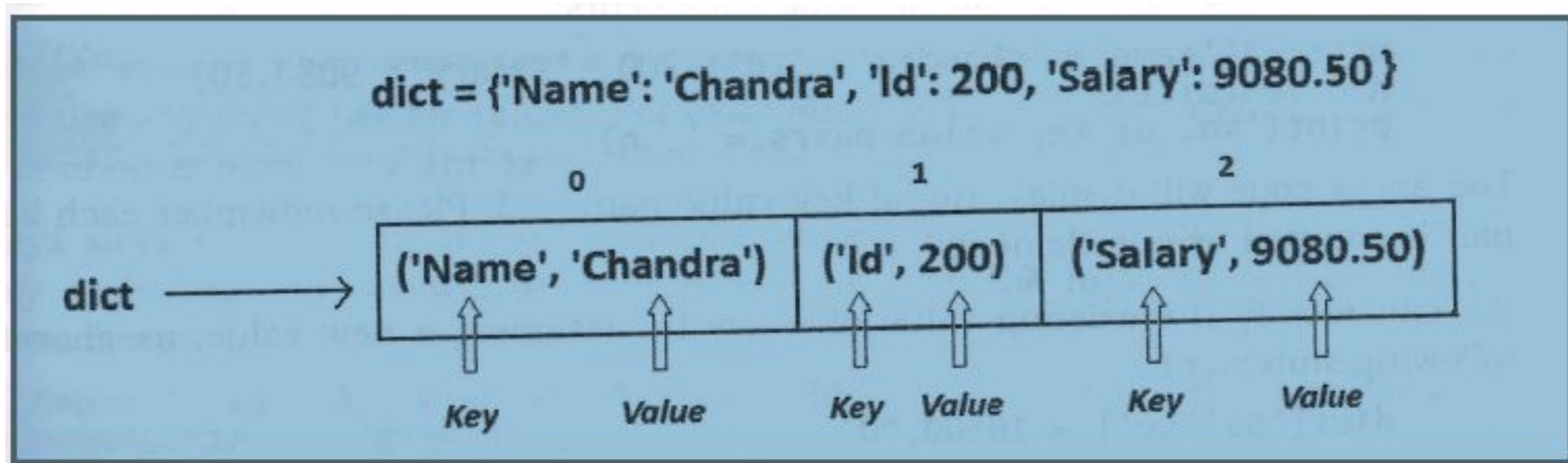


INTRODUCTION

- A dictionary represents a group of elements arranged in the form of key-value pairs.
- In the dictionary, the first element is considered as 'key' and the immediate next element is taken as its 'value'.
- The key and its value are separated by a colon “ : ”
- All the key-value pairs in a dictionary are inserted in curly braces { }
- To use this kind of structure, python provides *dict* built in class
- For example

```
>>> dict = {'Name': 'Chandra', 'Id': 200, 'Salary': 9080.50}
```





- [Go to Jupyter notebook for example](#)



OPERATIONS ON DICTIONARIES

- To access the elements of a dictionary, we **should not use *indexing or slicing***. For example, `dict[0]` or `dict[1:3]` etc. expressions will give error.
- To access the value associated with a key, we can mention the key name inside the square braces, as: `dict['Name']`. This will return the value associated with 'Name'. This is nothing but 'Chandra'.
- If we want to know how many key-value pairs are there in a dictionary, we can use the `len()` function, as shown in the following statements:

```
dict = {'Name': 'Chandra', 'Id': 200, 'Salary': 9080.50}
n = len(dict)
print('No. of key-value pairs =', n)
```
- The above code will display:
No. of key-value pairs = 3.
- Please remember each key-value pair is counted as one element.



- We can modify the existing value of a key by assigning a new value, as shown in the following statement:

```
dict['Salary'] = 10500.00
```

- Here, the 'Salary' value is modified as '10500.00'. The previous value of 'Salary', i.e. 9080.50 is replaced by the new value, i.e. 10500.00.
- We can also insert a new key-value pair into an existing dictionary. This is done by mentioning the key and assigning a value to it, as shown in the following statement:

```
dict['Dept'] = 'Finance'
```

- Now, if we display the dictionary using `print(dict)`, it will display:

```
{'Name': 'Chandra', 'Dept': 'Finance', 'Id': 200, 'salary': 10500.0}
```

- Observe the new pair 'Dept': 'Finance' is added to the dictionary.
- Also, observe that this pair is not added at the end of existing pairs. It may be added at any place in the dictionary. This is because the elements in dictionary and set are stored not in an order



- Suppose, we want to delete a key-value pair from the dictionary, we can use del statement as:

```
del dict['Id']
```

- This will delete the key 'Id' and its corresponding value from the dictionary. Now, the dictionary looks like this:

```
{'Name': 'Chandra', 'Dept': 'Finance', 'Salary': 10500.0}
```

- To test whether a 'key' is available in a dictionary or not, we can use 'in' and 'not in' operators. These operators return either True or False. Consider the following statement:

```
'Dept' in dict # check if 'Dept' is a key in dict
```

- The preceding statement will give:

```
True
```

- Now, if you write:

```
'Gender' not in dict # check if 'Gender' is not a key in dict
```

- Then the following output appears:

```
True
```



- We can use any datatypes for values.
- For example, a value can be a number, string, list, tuple or another dictionary. But keys should obey the following rules:
 - ❖ Keys should be unique. It means, duplicate keys are not allowed. If we enter same key again, the old key will be overwritten and only the new key will be available

Consider the following example:

```
>>> emp = {'Nag':10, 'vishnu':20, 'Nag':30}    # Key 'Nag' entered twice
```

```
>>> print(emp)
```

The output appears as:

```
>>> {'Nag': 30, 'Vishnu': 20} # first 'Nag' is replaced by new key and its value
```

- ❖ Keys should be immutable type. For example, we can use a number, string or tuples as keys since they are immutable. We **cannot use lists or dictionaries as keys**. If they are used as keys, we will get 'TypeError'.

Consider the following example:

```
>>> emp = { ['Nag']:10, 'Vishnu':20, 'Raj':30} # ['Nag'] is a list element
```

- so error

Traceback (most recent call last):

File "<pyshell#12>", line 1, in <module>

emp = {['Nag']:10, 'vishnu':20, 'Raj':30}

TypeError: unhashable type: 'list'



DICTIONARY METHODS

Method	Example	Description
clear()	d.clear()	Removes all key-value pairs from dictionary 'd'.
copy()	d1= d.copy()	Copies all elements from 'd' into a new dictionary 'd1'.
fromkeys()	d.fromkeys(s [,v])	Create a new dictionary with keys from sequence 's' and values all set to 'v'.
get()	d.get(k [,v])	Returns the value associated with key 'k'. If key is not found, it returns 'v'.
items()	d.items()	Returns an object that contains key-value pairs of 'd'. The pairs are stored as tuples in the object.
keys()	d.keys()	Returns a sequence of keys from the dictionary 'd'.
values()	d.values()	Returns a sequence of values from the dictionary 'd'.
update()	d.update(x)	Adds all elements from dictionary 'x' to 'd'.
pop()	d.pop(k [,v])	Removes the key 'k' and its value from 'd' and returns the value. If key is not found, then the value 'v' is returned. If key is not found and 'v' is not mentioned then 'KeyError' is raised.
setdefault()	d.setdefault(k [,v])	If key 'k' is found, its value is returned. If key is not found, then the k, v pair is stored into the dictionary 'd'



```
Python 3.8.5 Shell
File Edit Shell Debug Options Window Help

>>> d={1:10,'a':20}
>>> c={'b':30}
>>> d
{1: 10, 'a': 20}
>>> d.update(c)
>>> d
{1: 10, 'a': 20, 'b': 30}
>>> d.get('a')
20
>>> d.get('asdf',40)
40
>>> d.keys()
dict_keys([1, 'a', 'b'])
>>>
```

- [Go to Jupyter notebook for more examples](#)



USING FOR LOOP WITH DICTIONARIES

- For loop is very convenient to retrieve the elements of a dictionary
- Example program to **Find the Occurrences of Each Letter of a String using a Dictionary**

x	Found in dict	dict.get(x, 0) + 1	dict[x] = dict.get(x, 0) + 1	dict
B	No	1	dict['B'] = 1	{'B':1}
o	No	1	dict['o'] = 1	{'B':1, 'o':1}
o	Yes	2	dict['o'] = 2	{'B':1, 'o':2}
k	No	1	dict['k'] = 1	{'B':1, 'o':2, 'k':1}

- [Go to Jupyter notebook for examples](#)



SORTING THE ELEMENTS OF A DICTIONARY USING LAMBDA

- A lambda is a function that does not have a name.
- Lambda functions are written using a single statement and hence look like expressions.
- Lambda functions are written without using 'def' keyword. They are useful to perform some calculations or processing easily.
- For example,

```
f = lambda x, y: x+y
```

- The above expression is a lambda function with 2 arguments, x and y. After colon (:), we wrote the body, i.e. x+y. this is the value returned by the lambda function. At the time of calling this function, we are supposed to pass 2 values for x and y as: f(10, 15). This will return 25 as result.
- Let us see Python program to sort the elements of a dictionary based on a key or value using lambda
- [Go to Jupyter notebook for examples](#)



CONVERTING LISTS INTO DICTIONARY

- When we have two lists, it is possible to convert them into a dictionary
- The `zip()` function is useful to convert the sequences into a zip class object. There may be 1 or more sequences that can be passed to `zip()` function.
- The second step is to convert the zip object into a dictionary by using `dict()` function.
- [Go to Jupyter notebook for examples](#)



CONVERTING STRINGS INTO DICTIONARY

- When a string is given with key and value pairs separated by some delimiter (or separator) like a comma (,) we can convert the string into a dictionary
- Let us take an example string

str="Vijay=23 , Ganesh=20, Lakshmi=19, Nikhil=22"

- First we should split the string into pieces where a comma is found using split(',') method and then break the string at equals (=) symbol. This can be done using a for loop as:

for x in str.split(',')

y=x.split('=')

- Each piece of the string is available in 'y'. The second step is to store these pieces into a list 'lst' using append() method as:

lst . append(y)

- The third step is to convert the list into a dictionary 'd' using dict() function as:

d = dict(lst)

- Now, this dictionary 'd' contains the elements as:

{'Vijay': '23', 'Ganesh': '20', 'Lakshmi': '19', 'Nikhil': '22'}

- [Go to Jupyter notebook for examples](#)

